

Archivos Binarios, `struct` y Registros de Longitud Fija

Semana 8 — Persistencia binaria de datos, acceso directo a registros y mantenimiento de archivos (ABM)

1. ¿Por qué archivos binarios?

En la Semana 6 introdujimos los archivos de texto y un patrón básico de persistencia: abrir, leer línea por línea o escribir con `write`, y cerrar. Los archivos de texto son cómodos para datos destinados al ojo humano —listados, configuraciones, logs, CSV sencillos— pero resultan ineficientes y limitados cuando los datos son *datos estructurados para el programa*, no para el lector. Tres limitaciones importantes se hacen evidentes cuando un archivo de texto pasa de ser consultado a ser *procesado* intensivamente:

- **Costo de conversión:** cada número en un archivo de texto está representado como una secuencia de caracteres dígito. Para usarlo, hay que invocar `int()` o `float()` al leer, y `str()` al escribir. Sobre millones de registros, esas conversiones dominan el tiempo de ejecución.
- **Ocupación variable:** el número 42 ocupa 2 caracteres, pero 1234567890 ocupa 10. Esto hace imposible predecir dónde comienza el k -ésimo dato sin leer todo lo anterior. Los archivos de texto son, por naturaleza, de **acceso secuencial**.
- **Ausencia de tipos:** todo es cadena. La semántica —¿es un entero, un flotante, una fecha?— vive solo en la cabeza del programador y en los convertidores ad-hoc.

Los **archivos binarios** resuelven estas tres limitaciones a la vez: almacenan los bytes *tal cual están en memoria*, sin conversión a caracteres. Un entero de 4 bytes ocupa siempre 4 bytes, sea el valor 0, 42 o 2 147 483 647. Esa **regularidad en el tamaño** es la llave maestra que abre la puerta al **acceso directo**: si todos los registros miden lo mismo, el registro número k comienza exactamente en el byte $k \times \text{tamaño_registro}$, y se llega a él con una sola llamada del sistema operativo. Lo que con archivos de texto costaba $O(n)$ —porque había que leer desde el principio hasta encontrarlo— con archivos binarios cuesta $O(1)$.

Conexión con semanas anteriores

En la **Semana 2** aprendimos que los números enteros y flotantes se representan como secuencias de bits en memoria: complemento a dos para los enteros con signo, IEEE 754 para los flotantes. Esa representación binaria —que en un archivo de texto se *convierte* a dígitos decimales— es exactamente lo que los archivos binarios almacenan sin tocar. En la **Semana 5** vimos la búsqueda binaria sobre una lista ordenada en memoria, con acceso $O(1)$ al elemento k -ésimo. Los registros de longitud fija trasladan esa misma propiedad del arreglo en RAM al archivo en disco: el archivo binario se comporta como un *arreglo persistente*. En la **Semana 7** discutimos la gestión del heap y la reutilización de bloques liberados; hoy veremos que un archivo con *free list* reproduce ese mismo patrón a nivel del

sistema de archivos.

Comprender los archivos binarios y el módulo `struct` de Python nos permitirá:

- **Persistir** datos estructurados (registros con varios campos tipados) en un formato compacto y eficiente.
- **Acceder directamente** al k-ésimo registro sin recorrer los anteriores, combinando lo mejor del arreglo en memoria con la persistencia del disco.
- **Actualizar** un registro *in situ* sin reescribir todo el archivo.
- **Implementar un ABM** (Alta, Baja, Modificación) completo con reutilización del espacio liberado, mediante una *free list* encadenada.
- **Conectar** nuestra programación con el nivel del sistema operativo: bloques del sistema de archivos, llamada al sistema `lseek`, buffer cache, y *endianness*.

2. Archivos binarios vs. archivos de texto

Desde la perspectiva del programa, un archivo no es más que una secuencia de bytes. La diferencia entre un archivo «de texto» y un archivo «binario» no reside en los bytes que contiene —ambos son secuencias de bytes— sino en **cómo los interpretamos** y en **qué transformaciones aplica Python** al leer y al escribir.

2.1. Modos de apertura en binario

La sentencia `open()` acepta un parámetro de modo que distingue texto y binario:

Modo	Significado	Lectura devuelve	Escritura acepta
'r' / 'w' / 'a'	Texto	str (decodificado)	str
'rb' / 'wb' / 'ab'	Binario	bytes (crudos)	bytes
'r+b'	Binario lectura+escritura	bytes	bytes

¿Qué transformaciones hace Python en cada modo?

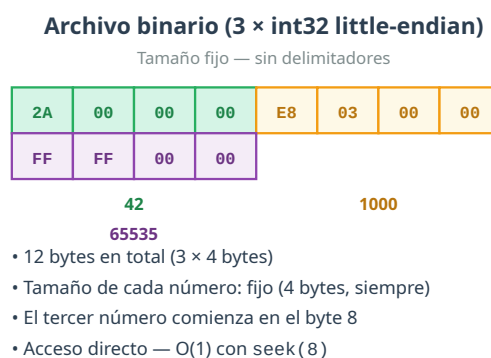
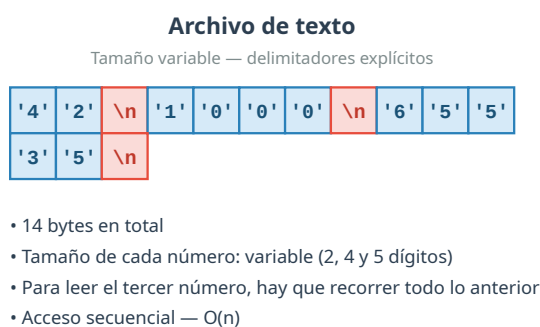
En modo **texto**, Python aplica dos transformaciones automáticas: (1) decodifica los bytes a caracteres usando una codificación (UTF-8 por defecto en sistemas Unix/Linux), y (2) traduce los finales de línea (por ejemplo, `\r\n` de Windows a `\n` al leer, y viceversa al escribir). En modo **binario** no hay ninguna transformación: se leen o se escriben los bytes *exactamente* como están en el archivo. Esto es indispensable cuando los bytes no representan texto —por ejemplo, un entero de 4 bytes que contiene el valor 0x0A en su primer byte se corrompería si Python lo interpretara como *newline*.

2.2. Ausencia de delimitadores de línea

Un archivo de texto es una sucesión de líneas separadas por `\n`. Un archivo binario **no tiene líneas**. Es una sucesión de bytes sin estructura impuesta por el formato: la estructura la define el programa que lee y escribe el archivo. Esto tiene una consecuencia directa: *no existe un equivalente a `readLine()` en archivos binarios*. El programa debe saber, de antemano, cuántos bytes leer para recuperar un dato. Lo que en texto era una *convención del formato* (la línea como unidad lógica) en binario se vuelve una *decisión de diseño del programador* (el registro como unidad lógica).

2.3. Visualización comparada

Veamos cómo se ven los mismos datos almacenados en ambos formatos. Supongamos que guardamos tres números enteros: 42, 1000 y 65535.



Lectura del diagrama

En el archivo binario, cada entero ocupa exactamente 4 bytes, sin importar su valor. El número 42 se almacena como 2A 00 00 00 (en little-endian, que veremos en la sección 3.3): el

byte menos significativo va primero. No hay separadores, no hay saltos de línea, no hay conversión a caracteres. El programa que lee sabe que cada registro mide 4 bytes, y con eso basta: el tercer entero comienza en el offset $2 \times 4 = 8$, al que se llega con una sola operación.

3. El módulo `struct`

Python, a diferencia de C, no expone directamente la representación binaria de sus tipos numéricos. Un `int` en Python es un objeto en el heap con tamaño variable (puede crecer arbitrariamente), no un campo de 4 u 8 bytes en el stack. Para convertir entre los objetos de alto nivel de Python (`int`, `float`, `str`) y las secuencias de bytes que se almacenan en archivos binarios, la biblioteca estándar provee el módulo **`struct`**.

El módulo `struct` ofrece tres funciones centrales:

Función	Propósito	Firma
<code>pack</code>	Empaqueta valores Python en bytes	<code>pack(formato, v1, v2, ...) → bytes</code>
<code>unpack</code>	Desempaqueta bytes en una tupla de valores Python	<code>unpack(formato, datos) → tuple</code>
<code>calcsize</code>	Calcula cuántos bytes ocupa un formato	<code>calcsize(formato) → int</code>

3.1. `pack` y `unpack`: los verbos del empaquetado

La operación `pack` toma una cadena de formato y uno o varios valores, y devuelve los bytes que representan esos valores según el formato. `unpack` hace la operación inversa: dado el mismo formato y una secuencia de bytes, devuelve una tupla con los valores decodificados. Son operaciones simétricas: *lo que se empaqueta con un formato debe desempaquetarse con el mismo formato*.

```
import struct
```

```
# Empaquetar: tres enteros de 4 bytes (formato '<iii' = little-endian, 3 int32
datos = struct.pack('<iii', 42, 1000, 65535)
print(datos)          # b'\x00\x00\x00\xe8\x03\x00\x00\xff\xff\x00\x00'
print(len(datos))     # 12  (3 × 4 bytes)

# Desempaquetar: el resultado es SIEMPRE una tupla
valores = struct.unpack('<iii', datos)
print(valores)        # (42, 1000, 65535)
```

```
print(type(valores)) # <class 'tuple'>
```

Observación clave: unpack devuelve una tupla

Incluso cuando se desempaqueta un solo valor, unpack devuelve una tupla de un elemento. Por eso es común ver la expresión (valor,) = struct.unpack('<i', datos) con la coma, o el acceso por índice struct.unpack('<i', datos)[0]. Esta convención es deliberada: pack/unpack están diseñados para trabajar con registros que agrupan varios campos, y una tupla es la forma natural de devolver múltiples valores.

3.2. Cadenas de formato

Una cadena de formato describe la estructura binaria del paquete. Cada carácter (a veces precedido por un número que indica repetición) representa un campo con un tipo y un tamaño fijos. Los caracteres de formato más frecuentes en un curso introductorio son:

Formato	Tipo C	Tipo Python	Tamaño (bytes)	Rango
b	signed char	int	1	−128 a 127
B	unsigned char	int	1	0 a 255
h	short	int	2	−32 768 a 32 767
H	unsigned short	int	2	0 a 65 535
i	int	int	4	−2 147 483 648 a 2 147 483 647
I	unsigned int	int	4	0 a 4 294 967 295
q	long long	int	8	$\pm 9,2 \cdot 10^{18}$
f	float (IEEE 754 simple)	float	4	$\pm 3,4 \cdot 10^{38}$ (~7 dígitos)
d	double (IEEE 754 doble)	float	8	$\pm 1,8 \cdot 10^{308}$ (~15 dígitos)
c	char	bytes de 1	1	0 a 255

Formato	Tipo C	Tipo Python	Tamaño (bytes)	Rango
Ns	char[N]	bytes de N	N	N bytes crudos (cadena)
Nx	relleno	—	N	N bytes de relleno (padding)
?	_Bool	bool	1	True / False

Convención sobre mayúsculas y minúsculas

La regla es simple y uniforme: **minúscula = con signo, mayúscula = sin signo**. `i` es un entero con signo de 4 bytes (rango negativo y positivo); `I` es un entero sin signo de 4 bytes (solo positivo, doble del rango). Usar mayúscula cuando el dominio no admite negativos duplica el rango de valores representables.

Conexión con la Semana 2: la representación binaria es la misma

Los tipos que lista la tabla no son «tipos de Python» ni «tipos de struct»: son los **tipos binarios** que estudiamos en la Semana 2. `i` corresponde a un entero con signo de 32 bits en *complemento a dos*. `f` corresponde a un flotante de 32 bits en *IEEE 754 simple precisión*, con su bit de signo, sus 8 bits de exponente y sus 23 bits de mantisa. `struct` es simplemente la puerta que nos permite escribir y leer esos formatos desde Python; la semántica de los bits ya la conocíamos.

El formato `Ns` merece una mención aparte porque se comporta de manera particular. A diferencia del resto, donde el número multiplica el campo (`3i` son tres `int32`), en `Ns` el número es *el tamaño total de un único campo de bytes*. Además:

- Si al empaquetar se pasa un bytes **más corto** que `N`, se rellena con bytes nulos (`\x00`) al final.
- Si se pasa uno **más largo**, se trunca silenciosamente a `N` bytes.
- Al desempaquetar, se devuelven exactamente `N` bytes, *incluyendo los ceros de relleno*. El programa debe quitarlos explícitamente si son indeseados (con `.rstrip(b'\x00')`).

```
import struct
```

```
# El campo nombre ocupa siempre 10 bytes, rellenos con \x00 si hace falta
paq = struct.pack('<10s', b'Ada')
print(paq)           # b'Ada\x00\x00\x00\x00\x00\x00\x00\x00'
```

```

print(len(paq))      # 10

(crudos,) = struct.unpack('<10s', paq)
print(crudos)        # b'Ada\x00\x00\x00\x00\x00\x00\x00'

# Para recuperar la cadena limpia, hay que quitar los ceros y decodificar
nombre = crudos.rstrip(b'\x00').decode('utf-8')
print(nombre)        # 'Ada'

```

Cuidado con cadenas en UTF-8 y *truncamiento por bytes*

Los campos `ns` se miden en **bytes**, no en caracteres. En UTF-8 un carácter acentuado como á ocupa 2 bytes, y un emoji puede ocupar 4. Un campo `10s` puede caber 10 letras ASCII (10 bytes) pero solo 5 letras acentuadas. Peor aún: si el truncamiento corta un carácter multi-byte por la mitad, la secuencia resultante no es UTF-8 válida y `decode('utf-8')` lanzará una excepción. En un diseño riguroso conviene validar la longitud en bytes antes de empaquetar, o usar `encode('utf-8')[:N]` con cuidado y la opción `errors='ignore'` al decodificar.

3.3. Endianness: el orden de los bytes

Los números multi-byte (cualquier cosa más allá de `b`, `B`, `c` y `?`) pueden almacenarse en dos órdenes: **little-endian**, con el byte menos significativo primero; o **big-endian**, con el byte más significativo primero. Esta decisión depende del hardware: los procesadores x86 y ARM (lo que corre en la inmensa mayoría de nuestras computadoras y teléfonos) son little-endian; los protocolos de red de Internet, en cambio, usan big-endian (por eso también se lo llama *network byte order*).

El entero 0x12345678 (decimal 305 419 896) en los dos órdenes de bytes



El módulo `struct` nos permite especificar el orden de bytes con un carácter al comienzo de la cadena de formato:

Prefijo	Orden de bytes	Alineación	Uso típico
@ (por defecto)	Nativo de la máquina	Con <i>padding</i> nativo	Estructuras internas del proceso (no recomendado para archivos)
=	Nativo de la máquina	Sin <i>padding</i>	Como el nativo, pero sin relleno automático
<	Little-endian	Sin <i>padding</i>	Formato portable pensado para x86/ARM
>	Big-endian	Sin <i>padding</i>	Formato portable pensado para protocolos/documentos
!	Big-endian (<i>network byte order</i>)	Sin <i>padding</i>	Protocolos de red (equivalente a >)

Regla práctica: no uses el prefijo por defecto en archivos

El prefijo por defecto @ no solo hereda la *endianness* del hardware —lo que hace el archivo no portable entre arquitecturas distintas— sino que también introduce *padding* nativo entre campos para alinearlos según las reglas del procesador, con lo que el tamaño real puede diferir de la suma de los tamaños declarados. En archivos binarios **siempre hay que elegir explícitamente un prefijo portable**: <, > o !. En este curso usaremos < (little-endian) por consistencia con la arquitectura habitual de nuestras máquinas y porque es la convención de la mayoría de los formatos modernos.

3.4. struct.calcsize: cuánto ocupa un formato

Antes de leer un registro de un archivo binario necesitamos saber *cuántos bytes leer*. Escribir ese número a mano en el código es frágil: si algún día cambia el formato del registro, el número queda obsoleto y el programa lee datos corruptos sin dar error. La función `struct.calcsize` elimina esa fuente de errores calculando el tamaño a partir del propio formato.

```
import struct

FORMATO_REGISTRO = '<i32s16s40s' # id, nombre, teléfono, email
TAM_REGISTRO      = struct.calcsize(FORMATO_REGISTRO)

print(TAM_REGISTRO) # 92 - 4 + 32 + 16 + 40
```

Patrón: el formato como única fuente de verdad

Definir `FORMATO_REGISTRO` como constante y derivar `TAM_REGISTRO` con `calcsize` es un ejemplo del principio *DRY (Don't Repeat Yourself)*: hay un solo lugar donde se describe la estructura del registro, y todas las operaciones (`pack`, `unpack`, `seek`, cálculo de cantidad de registros) se construyen a partir de esa descripción. Si mañana el campo `email` pasa a ser `60s`, cambiar la constante basta para que el resto del programa siga funcionando correctamente.

Calcular la cantidad de registros que hay en un archivo es otra aplicación típica de `calcsize`: basta con dividir el tamaño total del archivo por el tamaño de un registro. La función `os.path.getsize(ruta)` devuelve el tamaño en bytes:

```
import os
import struct

FORMATO_REGISTRO = '<i32s16s40s'
TAM_REGISTRO      = struct.calcsize(FORMATO_REGISTRO)

def cantidad_registros(ruta):
    """Devuelve la cantidad de registros de longitud fija en el archivo.

    Precondición: el archivo existe y su tamaño es múltiplo de TAM_REGISTRO.
    Postcondición: devuelve un entero no negativo.
    """
    tam_archivo = os.path.getsize(ruta) # bytes totales
    return tam_archivo // TAM_REGISTRO # división entera
```

4. Registros de longitud fija

La idea central de esta semana es la siguiente: si todos los registros de un archivo tienen

exactamente el mismo tamaño, el archivo se comporta como un **arreglo en disco**. Podemos acceder al k-ésimo registro en tiempo constante, podemos leerlo sin afectar a los vecinos, y podemos actualizarlo escribiendo exactamente su tamaño en su posición exacta, sin necesidad de reescribir nada más. El costo de esa propiedad es aceptar que cada registro ocupe lo mismo: los campos de texto se completan con bytes nulos hasta el máximo previsto.

4.1. Diseño del formato del registro

Diseñar un formato de registro implica tres decisiones:

- 1. **Qué campos** componen el registro.
- 2. **De qué tipo binario** es cada campo (int32, int16, float64, Ns, etc.).
- 3. **En qué orden** aparecen.

El orden importa para la alineación y la legibilidad, no para la corrección: cualquier orden es válido mientras lectura y escritura usen el mismo formato. Los tipos se eligen según el dominio del dato: una edad cabe en `u` (0-255), un ID de cliente cabe sobrado en `i`, y un nombre puede requerir 32 o 50 bytes según la aplicación. Un diseño apretado ahorra espacio; un diseño generoso reduce el riesgo de truncamiento.

4.2. Ejemplo: una agenda de contactos

Trabajemos con una agenda de contactos. Cada registro contiene un identificador numérico y tres cadenas: nombre, teléfono y email.

Campo	Tipo binario	Tamaño	Formato	Comentario
id	int32 con signo	4 bytes	i	Identificador único del contacto
nombre	bytes de 32	32 bytes	32s	Nombre completo, UTF-8, padded con \x00
telefono	bytes de 16	16 bytes	16s	Teléfono en formato libre
email	bytes de 40	40 bytes	40s	Dirección de correo electrónico
Total		92 bytes	'<i32s16s40s'	
Formato completo con little-endian				

Layout del registro — 92 bytes en total



Formato: <i32s16s40s → TAM_REGISTRO = 92 bytes

El registro k-ésimo comienza en el offset $k \times 92$

Con esta definición podemos escribir las funciones centrales de empaquetado y desempaquetado. Observá cómo cada función tiene una única responsabilidad y cómo el formato es la única fuente de verdad:

```
import struct
```

```
# Constantes del formato — única fuente de verdad
```

```
FORMATO_REGISTRO = '<i32s16s40s'
```

```
TAM_REGISTRO      = struct.calcsize(FORMATO_REGISTRO) # 92
```

```
TAM_NOMBRE        = 32
```

```
TAM_TELEFONO      = 16
```

```
TAM_EMAIL         = 40
```

```
def empaquetar_contacto(id, nombre, telefono, email):
```

```
    """Empaqueta un contacto en bytes con el formato del registro.
```

```
    Precondición: id es un int32 válido; nombre, telefono, email son str
                  cuyos equivalentes en UTF-8 caben en TAM_NOMBRE,
                  TAM_TELEFONO y TAM_EMAIL bytes respectivamente.
```

```
    Postcondición: devuelve un objeto bytes de exactamente TAM_REGISTRO.
```

```
    """
```

```
    nombre_b = nombre.encode('utf-8') # str → bytes
```

```
    telefono_b = telefono.encode('utf-8')
```

```
    email_b = email.encode('utf-8')
```

```
    return struct.pack(FORMATO_REGISTRO, id, nombre_b, telefono_b, email_b)
```

```
def desempaquetar_contacto(datos):
```

```
    """Desempaqueta un registro de TAM_REGISTRO bytes en una tupla legible.
```

```
    Precondición: datos es un bytes de longitud TAM_REGISTRO.
```

```
    Postcondición: devuelve (id, nombre, telefono, email) con los campos
                  de texto convertidos a str UTF-8, sin los \x00 de relleno.
```

```
    """
```

```
    id, nombre_b, telefono_b, email_b = struct.unpack(FORMATO_REGISTRO, datos)
```

```
    nombre = nombre_b.rstrip(b'\x00').decode('utf-8')
```

```
telefono = telefono_b.rstrip(b'\x00').decode('utf-8')
email     = email_b.rstrip(b'\x00').decode('utf-8')
return id, nombre, telefono, email
```

Patrón: las funciones pack/unpack como *capa de traducción*

Las funciones `empaquetar_contacto` y `desempaquetar_contacto` forman una **capa de traducción** entre el mundo de los objetos Python y el mundo de los bytes. Toda interacción con el archivo pasa por esta capa: nadie más en el programa necesita conocer la cadena de formato, ni hacer `encode/decode`, ni preocuparse por el *padding*. Esta separación de responsabilidades —también llamada *encapsulamiento*— conecta con la discusión de cohesión y acoplamiento de la Semana 4.

4.3. Acceso directo con `seek`

Cada archivo abierto tiene asociado un **cursor** o **puntero de archivo**: la posición dentro del archivo desde la que las próximas operaciones de lectura o escritura actuarán. Tres métodos manejan ese cursor:

Método	Propósito	Detalle
<code>archivo.tell()</code>	Consulta la posición actual del cursor	Devuelve un entero: el offset en bytes desde el inicio
<code>archivo.seek(offset)</code>	Mueve el cursor a una posición absoluta	Offset en bytes desde el inicio del archivo (por defecto)
<code>archivo.seek(desp, desde)</code>	Mueve el cursor de forma relativa	desde: 0 inicio (default), 1 posición actual, 2 fin

Con `seek` y la constante `TAM_REGISTRO`, el acceso al *k*-ésimo registro es un cálculo aritmético directo:

Archivo como arreglo de registros de longitud fija



```
archivo.seek(2 * TAM_REGISTRO)
```

El cursor salta directamente al Registro 2 sin leer los anteriores

La función para leer el registro k-ésimo se escribe en cinco líneas y es $O(1)$ sin importar cuán grande sea el archivo:

```
def leer_contacto(archivo, k):
    """Lee el registro k-ésimo del archivo y lo devuelve como tupla.

    Precondición: archivo está abierto en modo 'rb' o 'r+b'; k es un
                    índice válido ( $0 \leq k < \text{cantidad\_registros}$ ).
    Postcondición: el cursor queda posicionado al final del registro leído;
                    devuelve (id, nombre, telefono, email).
    """
    archivo.seek(k * TAM_REGISTRO)      # posición del k-ésimo
    datos = archivo.read(TAM_REGISTRO)   # exactamente 92 bytes
    return desempaquetar_contacto(datos)
```

4.4. Escritura, actualización *in situ* y creación del archivo

Para **crear** un archivo y agregarle contactos se abre en modo 'wb' (escritura binaria, que borra el contenido previo si existía) y se llama a `write` con los bytes del registro. Para **agregar** al final de un archivo existente, se usa 'ab'. Para **actualizar un registro en su lugar**, se abre en 'r+b' (lectura y escritura sin borrar lo anterior), se hace `seek` a la posición y se escribe encima.

```
def crear_agenda(ruta, contactos):
    """Crea un archivo binario nuevo con los contactos indicados.

    Precondición: contactos es una lista de tuplas (id, nombre, telefono, email)
    Postcondición: el archivo en 'ruta' contiene len(contactos) registros
                    consecutivos de longitud fija. Si ya existía, se sobrescribe.
    """
    with open(ruta, 'wb') as archivo:      # 'wb': crea o sobrescribe
        for id, nombre, telefono, email in contactos:
            datos = empaquetar_contacto(id, nombre, telefono, email)
            archivo.write(datos)

def actualizar_email(ruta, k, email_nuevo):
    """Reemplaza el email del registro k-ésimo sin tocar los demás campos.

    Precondición: ruta apunta a un archivo válido;  $0 \leq k < \text{cantidad}$ .
    Postcondición: el registro k-ésimo queda con el nuevo email; los demás
                    registros del archivo no se modifican.
    """
```

```

with open(ruta, 'r+b') as archivo:      # 'r+b': actualiza in situ
    id, nombre, telefono, _ = leer_contacto(archivo, k)
    datos = empaquetar_contacto(id, nombre, telefono, email_nuevo)
    archivo.seek(k * TAM_REGISTRO)      # volver a la posición del r
    archivo.write(datos)                 # sobrescribe exactamente !

```

¿Por qué el modo 'r+b' y no 'w+b'?

Aunque ambos modos permiten leer y escribir, hay una diferencia crucial: 'w+b' **borra el contenido del archivo al abrirlo** (como 'w'), mientras que 'r+b' **preserva el contenido** y permite modificarlo puntualmente. Para actualizaciones *in situ* sobre un archivo existente, 'r+b' es siempre la opción correcta. Una confusión con estos modos puede destruir instantáneamente todos los datos del archivo.

4.5. Verificación con volcado hexadecimal

Una manera poderosa de entender qué está guardado en un archivo binario es observar directamente sus bytes. Las herramientas `xxd` (estándar en sistemas Unix y también disponible en Git Bash para Windows) y `hexdump -C` muestran el contenido del archivo en hexadecimal junto con su interpretación ASCII. Supongamos que creamos un archivo con el contacto (1, "Ada", "4567-1122", "ada@fi.uba.ar"):

```

00000000: 0100 0000 4164 6100 0000 0000 0000 0000  ....Ada.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  .....
00000020: 0000 0000 3435 3637 2d31 3132 3200 0000  ....4567-1122...
00000030: 0000 0000 6164 6140 6669 2e75 6261 2e61  ....ada@fi.uba.a
00000040: 7200 0000 0000 0000 0000 0000 0000 0000  r.....
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  .....

```

Lectura del volcado

Los primeros 4 bytes 01 00 00 00 son el `id = 1` en little-endian (el byte menos significativo primero). A continuación hay 32 bytes del campo nombre: «Ada» en ASCII (41 64 61) seguido de 29 bytes de relleno `\x00`. Del offset 0x24 (36 en decimal) al 0x33 hay 16 bytes para el teléfono —«4567-1122» en ASCII y relleno—. Finalmente, del offset 0x34 (52 en decimal) al 0x5B hay 40 bytes para el email. Total: 4 + 32 + 16 + 40 = 92 bytes, que coincide con `TAM_REGISTRO`.

El volcado hexadecimal como herramienta de depuración

Cuando un programa que lee archivos binarios se comporta inesperadamente, el volcado hexadecimal muestra sin ambigüedad qué hay *realmente* en el archivo. Permite detectar *endianness* invertida, tamaños erróneos, datos truncados, o caracteres no válidos en campos UTF-8. Para archivos grandes se puede usar `xxd -s OFFSET -l LONGITUD archivo.bin` para inspeccionar un rango puntual.

5. Mantenimiento de archivos: el patrón ABM

En la vida real un archivo de datos no es solo un *dump* inicial: los registros nacen (*Alta*), mueren (*Baja*) y cambian (*Modificación*). El ciclo completo se conoce como **ABM** por las iniciales españolas, o CRUD (*Create, Read, Update, Delete*) en inglés. En un archivo de registros de longitud fija, cada operación tiene un diseño natural:

Operación	Implementación directa	Problema
Alta	Agregar el registro al final del archivo	El archivo solo crece
Baja	Desplazar todos los registros posteriores un lugar hacia atrás	$O(n)$ de E/S por cada baja; muy costoso
Modificación	seek y reescritura <i>in situ</i>	Sin problema: es la operación natural

La baja es la operación problemática. Desplazar físicamente los registros posteriores para «tapar el hueco» significa leer y reescribir potencialmente todo el archivo. Si borramos 100 registros de un archivo de 100 000, hacemos alrededor de 50 millones de operaciones de E/S. Además, un fallo a mitad del proceso dejaría el archivo en un estado corrupto.

5.1. Baja lógica: no borrar físicamente

La solución estándar es la **baja lógica**: en lugar de eliminar físicamente el registro, se lo *marca* como borrado con un *flag* en el propio registro. El espacio queda reservado pero inaccesible para las consultas: quien recorre el archivo saltea los registros marcados. Redefinimos el formato del registro agregándole un byte al comienzo:

Campo	Tipo	Tamaño	Propósito
activo	b (uint8)	1 byte	1 = registro activo; 0 = registro borrado
siguiente_libre	i (int32)	4 bytes	Índice del siguiente registro libre, o -1

Campo	Tipo	Tamaño	Propósito
id	i (int32)	4 bytes	Identificador del contacto
nombre	32s	32 bytes	Nombre del contacto
telefono	16s	16 bytes	Teléfono
email	40s	40 bytes	Dirección de correo
Total		97 bytes	Formato '<Bi i32s16s40s'

¿Por qué dos campos de metadato (activo y siguiente_libre)?

El byte activo se usa durante los recorridos normales: si vale 0, el registro se saltea. El campo siguiente_libre se usa solo cuando el registro está borrado, y sirve para *enlazar* los registros libres entre sí formando una **lista enlazada de espacio libre** (*free list*). Así, cuando se da un alta podemos reutilizar un espacio previamente liberado en tiempo $O(1)$, sin recorrer el archivo buscando huecos.

5.2. La *free list* encadenada

La idea de la free list es elegante: los registros borrados no son basura, son un *recurso a reutilizar*. Los mantenemos encadenados en una lista cuyo primer elemento está referenciado por una variable global (en RAM, mientras el programa corre, y en la cabecera del archivo, entre ejecución y ejecución). Cuando se da una baja, el registro se inserta al frente de la lista. Cuando se da un alta, se saca el primer registro de la lista si existe; si la lista está vacía, se agrega al final del archivo.

Free list: evolución del archivo tras tres operaciones

Estado 1: archivo con 5 registros, todos activos

Reg 0 activo=1	Reg 1 activo=1	Reg 2 activo=1	Reg 3 activo=1	Reg 4 activo=1
-------------------	-------------------	-------------------	-------------------	-------------------

cabeza_libre = -1 (lista vacía)

Estado 2: baja del registro 1

Reg 0 activo=1	Reg 1 (libre) activo=0, sig=-1	Reg 2 activo=1	Reg 3 activo=1	Reg 4 activo=1
-------------------	-----------------------------------	-------------------	-------------------	-------------------

cabeza_libre = 1

Estado 3: baja también del registro 3 (se enlaza al frente)

Reg 0 activo=1	Reg 1 (libre) activo=0, sig=-1	Reg 2 activo=1	Reg 3 (libre) activo=0, sig=1	Reg 4 activo=1
-------------------	-----------------------------------	-------------------	----------------------------------	-------------------

cabeza_libre = 3 → 1 → -1

(Un alta futura reusará el registro 3 primero)

La free list como lista enlazada sin memoria dinámica

Lo notable de esta estructura es que **la lista enlazada vive dentro del mismo archivo**: no necesitamos una estructura auxiliar en RAM ni asignación dinámica de memoria. Cada registro borrado usa el campo `siguiente_libre` —que ya estaba reservado pero que no se usaba mientras el registro estaba activo— para apuntar al índice del siguiente borrado. La cabeza de la lista se almacena en una cabecera al inicio del archivo. Este patrón *empalma* una lista enlazada con un arreglo: el arreglo es el conjunto de todos los registros, y la lista enlazada vive por sobre él usando índices en vez de punteros.

5.3. La cabecera del archivo

El valor `cabeza_libre` debe persistir entre ejecuciones del programa: si el archivo tiene registros borrados al cerrarse, la próxima vez que se abra el programa debe saber cuáles son reutilizables. La solución estándar es reservar una **cabecera de metadatos** al comienzo del archivo, antes del primer registro:

Campo de la cabecera	Tipo	Tamaño	Propósito
magic	4s	4 bytes	Identificador de formato (por ejemplo b'AGD1')
cantidad	i	4 bytes	Número total de registros (activos + borrados)
cabeza_libre	i	4 bytes	Índice del primer registro libre, -1 si no hay

Campo de la cabecera	Tipo	Tamaño	Propósito
Total		12 bytes	Formato '<4sii'

El *magic number* como sello del formato

Los primeros bytes del archivo llevan una constante reconocible —el *magic number*— que identifica sin ambigüedad el formato. Sistemas como Unix usan el comando `file` para reconocer tipos de archivo consultando estos sellos: un PDF empieza con %PDF, una imagen PNG con `\x89PNG\r\n`, un archivo ZIP con `PK\x03\x04`. En nuestro caso, `b'AGD1'` («agenda versión 1») nos previene de leer como agenda un archivo que no lo es, y nos permite evolucionar el formato: una futura versión usaría `b'AGD2'` y el programa podría decidir cómo procesar cada una.

Con la cabecera, el offset físico del registro k -ésimo dentro del archivo pasa a ser:

$$\text{offset_registro}(k) = \text{TAM_CABECERA} + k \times \text{TAM_REGISTRO} = 12 + k \times 97$$

5.4. Código completo del ABM

Con las piezas anteriores podemos escribir un ABM completo. El código que sigue respeta el modelo estructurado del curso: sección declarativa con las constantes, funciones bien documentadas, y sección algorítmica con prólogo, resolución y epílogo.

```
# =====
# Sección declarativa
# =====
import struct
import os

FORMATO_CABECERA = '<4sii' # magic, cantidad, cabez
TAM_CABECERA     = struct.calcsize(FORMATO_CABECERA) # 12
MAGIC            = b'AGD1'

FORMATO_REGISTRO = '<Bi i32s16s40s' # activo, siguiente_lib
TAM_REGISTRO     = struct.calcsize(FORMATO_REGISTRO) # 97

ACTIVO           = 1
```

```
BORRADO          = 0
FIN_LISTA        = -1
```

```
# =====:
# Funciones – cabecera
# =====:
```

```
def inicializar_archivo(ruta):
```

```
    """Crea un archivo nuevo con la cabecera vacía.
```

```
    Postcondición: el archivo tiene TAM_CABECERA bytes y ningún registro.
```

```
    """
```

```
    with open(ruta, 'wb') as archivo:
```

```
        cab = struct.pack(FORMATO_CABECERA, MAGIC, 0, FIN_LISTA)
```

```
        archivo.write(cab)
```

```
def leer_cabecera(archivo):
```

```
    """Lee la cabecera del archivo.
```

```
    Precondición: archivo abierto en 'rb' o 'r+b'.
```

```
    Postcondición: devuelve (magic, cantidad, cabeza_libre); el cursor
                    queda al final de la cabecera.
```

```
    """
```

```
    archivo.seek(0)
```

```
    datos = archivo.read(TAM_CABECERA)
```

```
    return struct.unpack(FORMATO_CABECERA, datos)
```

```
def escribir_cabecera(archivo, cantidad, cabeza_libre):
```

```
    """Actualiza la cabecera del archivo con los valores dados.
```

```
    Precondición: archivo abierto en 'r+b'.
```

```
    Postcondición: la cabecera queda actualizada; cursor al final de la cabecera
    """
```

```
    archivo.seek(0)
```

```
    archivo.write(struct.pack(FORMATO_CABECERA, MAGIC, cantidad, cabeza_libre))
```

```
# =====:
# Funciones – registros
# =====:
```

```
def offset_de(k):
```

```
    """Devuelve el offset físico del registro k-ésimo."""
```

```
    return TAM_CABECERA + k * TAM_REGISTRO
```

```

def leer_registro(archivo, k):
    """Lee el registro k-ésimo y lo devuelve como tupla.

    Precondición:  $0 \leq k < \text{cantidad de registros}$ .
    Postcondición: devuelve (activo, siguiente, id, nombre, telefono, email).
    """
    archivo.seek(offset_de(k))
    datos = archivo.read(TAM_REGISTRO)
    activo, sig, id, nom, tel, mail = struct.unpack(FORMATO_REGISTRO, datos)
    nombre = nom.rstrip(b'\x00').decode('utf-8')
    telefono = tel.rstrip(b'\x00').decode('utf-8')
    email = mail.rstrip(b'\x00').decode('utf-8')
    return activo, sig, id, nombre, telefono, email

def escribir_registro(archivo, k, activo, siguiente, id, nombre, telefono, ema
    """Escribe un registro completo en la posición k-ésima.

    Precondición: archivo abierto en 'r+b';  $0 \leq k \leq \text{cantidad}$  (permite append).
    Postcondición: los TAM_REGISTRO bytes que comienzan en offset_de(k)
                    quedan con los nuevos valores.
    """
    nombre_b = nombre.encode('utf-8')
    telefono_b = telefono.encode('utf-8')
    email_b = email.encode('utf-8')
    datos = struct.pack(FORMATO_REGISTRO, activo, siguiente, id, nombre_b, tel
    archivo.seek(offset_de(k))
    archivo.write(datos)

# =====
# Operaciones ABM
# =====

def alta(ruta, id, nombre, telefono, email):
    """Da de alta un nuevo contacto, reutilizando un registro libre si lo hay.

    Precondición: ruta apunta a un archivo válido (con cabecera).
    Postcondición: el contacto queda agregado al archivo; devuelve el
                    índice k donde quedó almacenado.
    """
    with open(ruta, 'r+b') as archivo:
        magic, cantidad, cabeza_libre = leer_cabecera(archivo)

        if cabeza_libre != FIN_LISTA:
            # Reusa un registro libre
            k = cabeza_libre

```

```

        _, siguiente_nuevo, _, _, _, _ = leer_registro(archivo, k)
        escribir_registro(archivo, k, ACTIVO, FIN_LISTA, id, nombre, telefono, email, siguiente_nuevo)
        escribir_cabecera(archivo, cantidad, siguiente_nuevo)
    else:
        # Agrega al final
        k = cantidad
        escribir_registro(archivo, k, ACTIVO, FIN_LISTA, id, nombre, telefono, email, siguiente_nuevo)
        escribir_cabecera(archivo, cantidad + 1, cabeza_libre)

    return k

def baja(ruta, k):
    """Da de baja (lógica) el registro k-ésimo.

    Precondición:  $0 \leq k < \text{cantidad}$ ; el registro está activo.
    Postcondición: el registro queda marcado como borrado y enlazado al
                    frente de la free list.

    """
    with open(ruta, 'r+b') as archivo:
        magic, cantidad, cabeza_libre = leer_cabecera(archivo)
        escribir_registro(archivo, k, BORRADO, cabeza_libre, 0, '', '', '')
        escribir_cabecera(archivo, cantidad, k)

def modificacion(ruta, k, id, nombre, telefono, email):
    """Modifica in situ el contenido del registro k-ésimo.

    Precondición:  $0 \leq k < \text{cantidad}$ ; el registro está activo.
    Postcondición: los campos del registro k-ésimo quedan actualizados;
                    activo y siguiente_libre no cambian.

    """
    with open(ruta, 'r+b') as archivo:
        escribir_registro(archivo, k, ACTIVO, FIN_LISTA, id, nombre, telefono, email, siguiente_nuevo)

# =====
# Consulta
# =====

def listar_activos(ruta):
    """Recorre el archivo y devuelve los contactos activos.

    Postcondición: devuelve una lista de tuplas (k, id, nombre, telefono, email)
                    con los registros cuyo flag activo es ACTIVO.

    """
    resultado = []
    with open(ruta, 'rb') as archivo:

```

```

_, cantidad, _ = leer_cabecera(archivo)
for k in range(cantidad):
    activo, _, id, nombre, telefono, email = leer_registro(archivo, k)
    if activo == ACTIVO:
        resultado.append((k, id, nombre, telefono, email))
return resultado

# =====
# Sección algorítmica – Prólogo / Resolución / Epílogo
# =====

# Prólogo: preparación del archivo
RUTA = 'agenda.dat'
inicializar_archivo(RUTA)

# Resolución: tres altas, una baja y una modificación
alta(RUTA, 1, 'Ada Lovelace', '4567-1122', 'ada@fi.uba.ar')
alta(RUTA, 2, 'Alan Turing', '4567-3344', 'alan@fi.uba.ar')
alta(RUTA, 3, 'Grace Hopper', '4567-5566', 'grace@fi.uba.ar')
baja(RUTA, 1) # borra a Turing
modificacion(RUTA, 2, 3, 'Grace Hopper', '4567-9999', 'grace@fi.uba.ar')

# Un alta posterior reutiliza el hueco de Turing (k=1)
k = alta(RUTA, 4, 'Edsger Dijkstra', '4567-7788', 'edsger@fi.uba.ar')
print(f'Dijkstra quedó en k = {k}') # k = 1 (reusó el hueco)

# Epílogo: listar y mostrar el estado final
for k, id, nombre, telefono, email in listar_activos(RUTA):
    print(f'[{k}] {id} {nombre} {telefono} {email}')

```

Patrón: el archivo como estructura de datos persistente

Observá el paralelo exacto con las listas enlazadas que estudiaremos en detalle en semanas futuras: la *cabeza* de la lista se guarda en un lugar conocido (aquí, la cabecera del archivo; en una lista en memoria, una variable), y cada nodo guarda el índice del siguiente nodo libre. La baja equivale a *push* al frente de la lista: $O(1)$. La reutilización en el alta equivale a *pop* del frente: también $O(1)$. No hace falta recorrer el archivo buscando huecos.

Las consecuencias del ABM y la limitación del borrado lógico

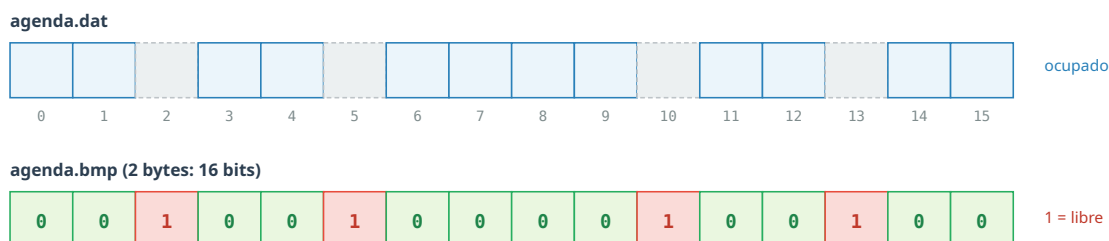
El borrado lógico **no libera espacio en disco**: el archivo crece hasta el pico histórico de registros activos y luego solo se reduce con una operación especial de *compactación* —crear un archivo nuevo copiando solo los registros activos, y reemplazar el original por el nuevo. Algunas bases de datos (como SQLite con `VACUUM`) ofrecen esta operación explícitamente. La decisión de cuándo compactar es un compromiso entre la velocidad de las operaciones individuales (que favorecen la free list) y el uso de disco (que favorece la compactación periódica).

5.5. Alternativa: bitmap de registros libres

La *free list* encadenada no es la única técnica posible para gestionar el espacio borrado. Existe al menos una alternativa igualmente válida que conviene conocer, porque es la que efectivamente usan los sistemas de archivos reales (ext4, NTFS, APFS) para gestionar sus propios bloques libres: un **bitmap** auxiliar.

La idea es sencilla: en un archivo paralelo —o en un bloque reservado dentro de la cabecera del mismo archivo— se guarda un bit por cada registro, que vale 1 si el registro está libre y 0 si está ocupado. Para una agenda de 1000 contactos alcanza con un bitmap de 125 bytes; para un millón, 125 KiB; para mil millones, 125 MiB. El espacio es modesto comparado con los datos que gobierna.

Bitmap de 16 registros: cada bit indica si el registro homólogo está libre



La manipulación de bits individuales en Python se hace con operadores aritméticos sobre enteros. Dado un índice de registro k , el bit correspondiente está en el byte $k // 8$ y dentro de éste, en la posición $k \% 8$. Las tres operaciones clave son:

```
def bit_libre(bitmap, k):
    """Devuelve True si el registro k está marcado como libre en el bitmap.

    Precondición: bitmap es un bytearray;  $0 \leq k < \text{len}(\text{bitmap}) * 8$ .
    """
    byte = bitmap[k // 8]                                # byte que contiene el bit
    return (byte >> (k % 8)) & 1 == 1                    # aislar el bit y comparar
```

```
def marcar_libre(bitmap, k):
    """Pone en 1 el bit k del bitmap (registro k pasa a libre)."""
    bitmap[k // 8] |= (1 << (k % 8))          # OR con la máscara del bit

def marcar_ocupado(bitmap, k):
    """Pone en 0 el bit k del bitmap (registro k pasa a ocupado)."""
    bitmap[k // 8] &= ~(1 << (k % 8))        # AND con la máscara negada

def buscar_slot_libre(bitmap):
    """Devuelve el índice del primer registro libre, o -1 si no hay ninguno."""
    n_registros = len(bitmap) * 8
    k = 0
    while k < n_registros and not bit_libre(bitmap, k):
        k += 1
    return k if k < n_registros else -1
```

El *alta* con reutilización busca un slot libre en el bitmap, escribe el registro en esa posición del archivo de datos y marca el bit como ocupado. La *baja* marca el bit como libre y pone el byte activo del registro en 0. El recorrido secuencial de registros activos no necesita consultar el bitmap en absoluto, porque usa el byte activo interno. El bitmap solo se consulta al buscar dónde insertar.

Mejora simple: cursor del último slot libre encontrado

La búsqueda lineal de un bit libre desde el principio del bitmap es $O(n/8)$. Una mejora barata es recordar en la cabecera el índice del último slot libre devuelto y empezar la próxima búsqueda desde allí, dando la vuelta al llegar al final. El costo amortizado se acerca a $O(1)$ bajo patrones de uso realistas, donde las bajas se distribuyen en el tiempo. Los sistemas de archivos como ext4 implementan variantes más sofisticadas (árboles B de extensiones libres) para volúmenes de TiB.

Comparación: free list encadenada vs. bitmap auxiliar

Aspecto	Free list encadenada	Bitmap auxiliar
Costo estructural permanente	4 bytes por registro (campo siguiente_libre)	1 bit por registro (~0,125 bytes)
Alta con reutilización	$O(1)$ — consume la cabeza	$O(n/64)$ peor caso; $O(1)$

Aspecto	<i>Free list</i> encadenada	Bitmap auxiliar
(tiempo)		amortizado con cursor
Baja (tiempo)	O(1)	O(1)
Arquitectura física	Un solo archivo autocontenido	Dos archivos a sincronizar (o cabecera interna)
Riesgo de corrupción	Si se pierde la cabeza, se pierde todo el espacio libre	Si se desincroniza, se corrompe la asignación
Estadísticas de espacio libre	Requiere recorrer la cadena — O(libres)	Contar bits en uno es <code>bin(byte).count('1')</code>
Uso real	Sistemas pequeños, estructuras ad-hoc	Sistemas de archivos (ext4, NTFS, APFS), DBMS

Ninguna de las dos técnicas es universalmente superior. La *free list* es imbatible por su simplicidad: un solo archivo, una sola cadena, cero sincronización. El bitmap es imbatible para volúmenes grandes y cuando se quieren estadísticas baratas («¿cuánto espacio libre hay?» respondería un sistema que use bitmap con un simple conteo de bits; uno que use *free list* tendría que recorrerla).

Curiosidad avanzada: `ctypes.Union` y estructuras C

En lenguajes como C existe una construcción llamada `union`: un mismo fragmento de memoria puede interpretarse como uno u otro tipo según un discriminador. El módulo `ctypes` de Python permite definir `ctypes.Structure` y `ctypes.Union` para interactuar con código C, y ofrece una forma muy expresiva de implementar el registro con *free list*: dos estructuras distintas —una para el registro activo, otra para el registro borrado con `siguiente_libre`— conviven en la misma región física vía `Union`. Esta técnica requiere definición de clases, que en este curso se introduce más adelante; queda aquí como puntero para quien quiera explorar cómo se escriben estos formatos en lenguajes de sistemas.

Desafío extracurricular

Reimplementar el ABM de la agenda usando un **archivo auxiliar de bitmap** en lugar de la *free list* encadenada. Consignas:

- Mantener el `byte activo` dentro del registro para no leer el bitmap durante recorridos secuenciales.
- Eliminar el campo `siguiente_libre`: el registro queda en 96 bytes sin ese campo, o 92 si

también se elimina `activo` (pero entonces cada recorrido secuencial forzaría a leer el bitmap en paralelo).

- Gestionar ambos archivos (`agenda.dat` y `agenda.bmp`) de forma consistente: si el programa se interrumpe entre la escritura del registro y la actualización del bit, ¿cómo se recupera la consistencia al reabrir?
- Implementar una operación `contar_libres()` que devuelva la cantidad de registros libres en tiempo $O(n/64)$ contando bits encendidos, y compararla con la versión equivalente de la *free list* (que necesita recorrerla).

6. Registros de longitud variable: dos escenarios

Los registros de longitud fija resuelven elegantemente el problema del acceso directo, pero pagan un precio: el espacio desperdiciado por el relleno. Si el campo nombre mide 32 bytes para acomodar los casos extremos, un contacto con nombre «Ana» desperdicia 29 bytes. Sobre millones de registros, el desperdicio puede ser significativo. Los **registros de longitud variable** eliminan ese desperdicio almacenando solo los bytes efectivamente necesarios, a cambio de perder la propiedad de acceso directo por aritmética simple.

El problema se bifurca en dos escenarios cualitativamente distintos según el uso. El tratamiento de esta semana es conceptual; la implementación completa queda como desafío extracurricular, y algunas de las técnicas reaparecerán en semanas futuras con herramientas más potentes.

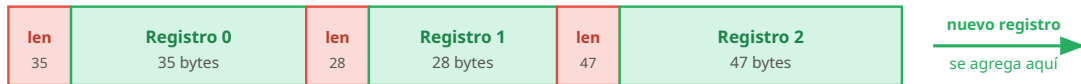
Dos escenarios, dos soluciones canónicas

- **Datos transaccionales:** registros que se agregan y nunca se modifican. Ejemplos: logs de eventos, transacciones bancarias, mediciones de sensores. La solución canónica es el *log append-only*.
- **Datos maestros:** registros que se crean, modifican y eventualmente borran —los catálogos del sistema, tablas de clientes, inventarios. La solución canónica son las *slotted pages*: bloques de tamaño fijo que contienen internamente registros variables.

6.1. Escenario transaccional: *log append-only*

En muchos sistemas, los datos se escriben una sola vez y no se modifican: un log de eventos, las transacciones confirmadas de una base de datos, los mensajes de un broker, las mediciones de un sensor. Para este patrón, la estructura más simple y eficiente es el **log append-only**: un archivo donde cada registro se agrega al final, nunca se borra ni se modifica, y cada registro lleva un **prefijo de longitud** que indica cuántos bytes siguen.

Log append-only: cada registro con su prefijo de longitud



Las altas son $O(1)$: solo `seek(0, 2)` al fin y `write(len + datos)`.

El recorrido es secuencial: se lee el prefijo, se leen los *len* bytes siguientes, se avanza al próximo prefijo.

Nunca hay fragmentación interna: cada registro ocupa exactamente lo que mide.

Este es el formato nativo de numerosos sistemas de producción: los *write-ahead logs* de PostgreSQL y SQLite, los segmentos de Apache Kafka, los archivos de log de servicios como Nginx o Apache en variante binaria (NCSA Common Log), los *event stores* de sistemas de arquitectura CQRS. La razón es doble: la escritura es la operación más frecuente y se hace en $O(1)$ al final del archivo; y la ausencia de modificaciones elimina toda la complejidad de fragmentación, compactación y sincronización.

Para recuperar el acceso directo al registro *k*-ésimo cuando hace falta, se acompaña el archivo de datos con un **archivo índice** separado que guarda el offset de cada registro:

Archivo	Contenido	Tamaño por entrada
eventos.log	Registros de longitud variable concatenados, con prefijo	Variable
eventos.idx	Offsets (int64) del inicio del registro <i>k</i> -ésimo	8 bytes fijos

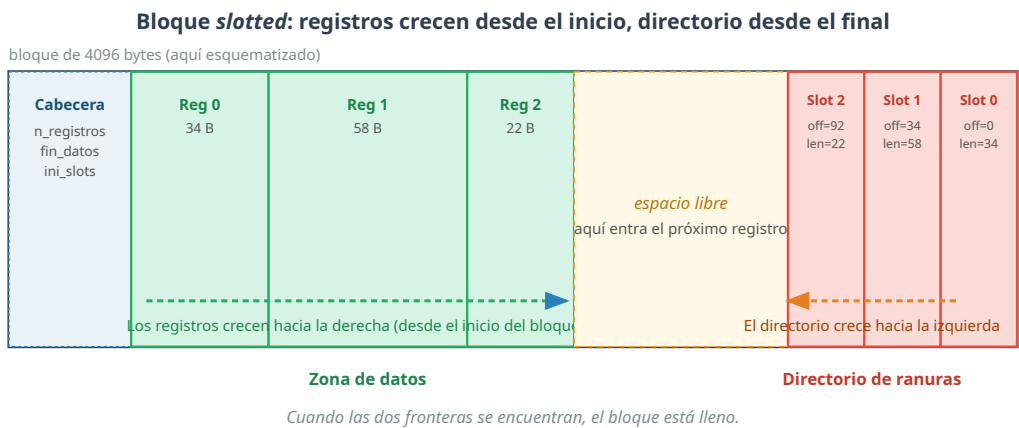
Cuando hace falta leer el registro *k*, se hace `seek(k · 8)` en el índice para obtener el offset, luego `seek(offset)` en el archivo de datos, y finalmente se lee el prefijo más los bytes. El índice se construye una vez al crear el archivo (cada `append` agrega una entrada al índice también) y se lee entero a memoria al abrir el archivo, si cabe.

6.2. Escenario maestro con ABM: *slotted pages*

El caso difícil es el de datos maestros con ABM y registros de longitud variable: un catálogo de productos donde las descripciones son de tamaño dispar, una tabla de clientes donde los nombres no entran en un campo fijo de 32 bytes sin desperdicio, un índice de documentos donde los textos varían de líneas a párrafos. En este escenario las altas, bajas y modificaciones pueden hacer crecer, achicar o eliminar registros en cualquier orden, y un archivo «plano» de registros variables concatenados se volvería rápidamente infragmentable: modificar un registro para que mida más forzaría a desplazar todos los registros siguientes.

La solución canónica se llama ***slotted pages*** —bloques con directorio de ranuras— y es exactamente la que usan PostgreSQL, SQL Server e InnoDB para organizar sus tablas en disco. La idea consiste en dividir el archivo de datos en **bloques de tamaño fijo** (típicamente 4 KiB u 8 KiB,

alineados con los bloques del sistema de archivos), y acomodar los registros variables *dentro* de cada bloque con la siguiente geometría:



El **directorio de ranuras** es una tabla al final del bloque con una entrada por cada registro almacenado, que contiene el offset y la longitud del registro dentro del bloque. Los registros identificados desde afuera no se nombran por posición dentro del archivo de datos, sino por el par (número de bloque, número de ranura). Este identificador compuesto se conserva a pesar de las modificaciones internas al bloque.

La geometría da tres propiedades interesantes:

- Una **modificación** que no aumenta el tamaño del registro se hace *in situ*. Si lo aumenta y ya no cabe en el bloque, el motor reubica el registro a otro bloque y deja una redirección (en jerga PostgreSQL, un *tuple update* con *HOT chain*).
- Una **baja** marca el slot como borrado en el directorio (o lo pone a offset=-1). El espacio no se reutiliza inmediatamente, pero el bloque puede ser *compactado* periódicamente desplazando registros vivos hacia el inicio.
- Un **alta** se acomoda en un bloque que tenga suficiente espacio libre. Para saber qué bloque elegir sin abrirlos todos, se usa un archivo auxiliar que registra el espacio libre de cada bloque.

El archivo auxiliar de espacio libre es estructuralmente análogo al bitmap de la Sección 5.5, pero ahora cada entrada es un int16 o int32 por bloque —espacio libre en bytes— en vez de un bit:

Archivo	Contenido	Tamaño por entrada
catalogo.dat	Bloques de 4096 bytes con registros variables y directorio de ranuras	4096 bytes
catalogo.fsm	Espacio libre en bytes de cada bloque, en la posición homóloga	2 bytes (uint16)

La sigla *fsm* corresponde al *Free Space Map* que efectivamente usa PostgreSQL. Al dar de alta un registro variable, el motor busca en el FSM un bloque cuyo espacio libre alcance, abre ese único bloque y lo inserta; no necesita recorrer el archivo de datos.

Patrón transversal: tabla auxiliar de bloque por bloque

La cabecera de cada bloque contiene su propio directorio de ranuras, pero el archivo de espacio libre vive *afuera* del archivo de datos. Es un patrón que retoma lo visto en la Sección 5.5 con el bitmap: cuando hay dos archivos paralelos en los que la posición *k*-ésima de uno habla del bloque *k*-ésimo del otro, se gana en flexibilidad (se puede consultar el auxiliar sin abrir el archivo grande) y se paga con el costo de sincronización entre ambos.

6.3. Desafíos extracurriculares

Propuesta 1 — Log append-only con índice

Implementar un *event store* que almacene eventos de longitud variable (nombre del evento + timestamp + payload JSON codificado como bytes) con prefijo de longitud y un archivo índice paralelo de offsets. Consignas:

- Operación `append(evento)`: agrega al final de ambos archivos en $O(1)$.
- Operación `leer(k)`: devuelve el *k*-ésimo evento en $O(1)$ usando el índice.
- Operación `recorrer()`: recorre el log en orden cronológico sin usar el índice, solo los prefijos de longitud.
- Operación `reconstruir_indice()`: a partir del archivo de datos únicamente, regenera el índice desde cero (útil si el índice se pierde o se corrompe).

Propuesta 2 — Slotted pages con Free Space Map

Implementar un catálogo de productos con descripciones de longitud variable usando *slotted pages* de 4096 bytes. Consignas:

- Cada bloque tiene cabecera (`n_registros`, `fin_datos`, `ini_slots`), zona de datos que crece desde el principio y directorio de ranuras que crece desde el final.
- Identificar cada producto por el par (`n_bloque`, `n_slot`).
- Mantener un archivo auxiliar FSM con el espacio libre por bloque.
- Implementar alta, baja lógica (slot con `offset=-1`) y modificación que quepa *in situ*. La modificación que no quepa en el bloque original puede devolver un error controlado o implementar reubicación.

- Implementar una operación `compactar_bloque(n)` que corra los registros vivos hacia el inicio del bloque y actualice el directorio.

7. Serialización de datos

Hasta aquí hemos usado `struct` para convertir valores Python en bytes y viceversa. A este proceso se lo llama en general **serialización**: traducir estructuras de datos en memoria a una representación lineal que pueda guardarse en un archivo o enviarse por la red. La operación inversa es la **deserialización**.

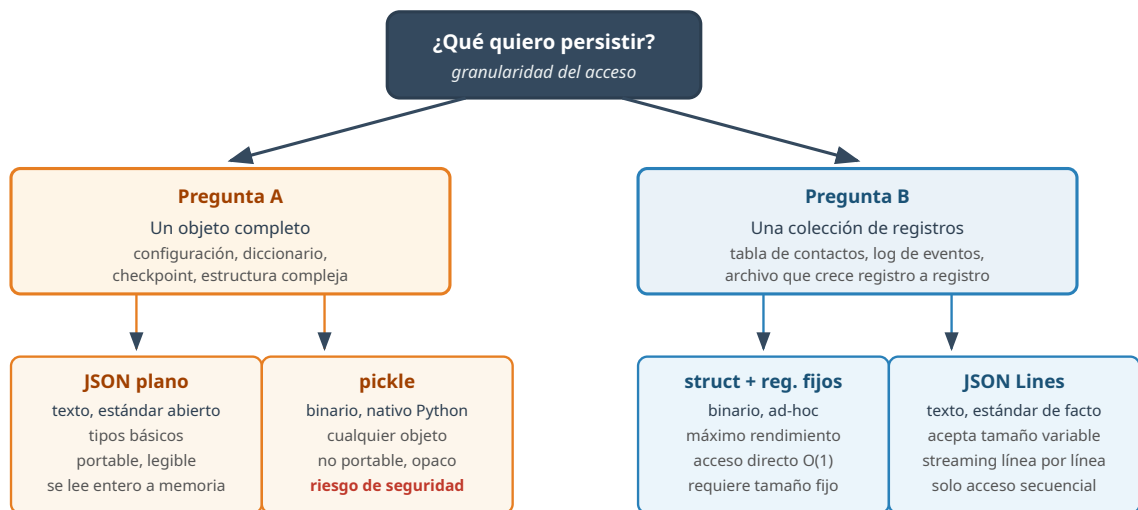
El módulo `struct` no es la única herramienta de serialización disponible en Python. La biblioteca estándar incluye otros dos módulos de uso muy difundido que cubren necesidades distintas, y que conviene presentar antes de profundizar en cada uno:

- `json` implementa el formato **JSON** (*JavaScript Object Notation*), un estándar abierto basado en texto que representa estructuras anidadas de tipos básicos —números, cadenas, booleanos, listas y diccionarios—. Es legible por humanos, portable entre lenguajes y seguro de deserializar.
- `pickle` es un formato binario nativo de Python que serializa *cualquier* objeto Python, incluyendo instancias de clases del usuario y estructuras complejas como árboles o grafos. Es muy expresivo, pero su mecanismo de deserialización permite la ejecución de código arbitrario, lo que lo hace inseguro para datos provenientes de fuentes no confiables.

Tener tres alternativas (`struct`, `json`, `pickle`) plantea naturalmente la pregunta de cuándo conviene cada una. La respuesta no se reduce a «binario o texto» ni a «estándar o nativo», sino a una distinción más profunda que organiza esta sección.

7.1. Dos preguntas distintas, dos familias de soluciones

La pregunta verdadera al elegir un formato de serialización es de qué **granularidad** será el acceso a los datos persistidos. Eso determina primero la familia de soluciones; las propiedades secundarias —binario o texto, estándar o nativo, compacto o verboso— recortan luego dentro de la familia ya elegida:



El eje que importa: granularidad del acceso

Pregunta A: ¿quiero persistir *un objeto completo* —una lista, un diccionario, una configuración, un checkpoint— para recuperarlo *entero* en otra ejecución del programa?

Pregunta B: ¿quiero persistir *una colección de registros* que puede ser grande, crecer con el tiempo, y sobre la que voy a hacer operaciones *registro por registro* (agregar uno, buscar uno, recorrerlos todos)?

Las dos preguntas tienen respuestas distintas. Mezclarlas conduce a diseños malos: usar JSON plano para una base de datos de un millón de registros fuerza a cargarla entera en memoria en cada apertura; usar struct con registros fijos para guardar un diccionario de configuración es usar una llave inglesa para apretar un tornillo.

Resumiendo el criterio:

Propósito	Técnicas apropiadas
Colección de registros, homogénea y de tamaño fijo, con acceso directo al k-ésimo	struct + registros fijos (lo central de esta semana)
Colección de registros, heterogénea o de tamaño variable, procesada secuencialmente	JSON Lines (también llamado NDJSON)
Un objeto completo para persistir entre ejecuciones (configuración, diccionario, lista)	JSON plano
Un objeto Python arbitrario (instancias, árboles, grafos) para uso interno del programa	pickle (con cuidados de seguridad)

7.2. struct y registros fijos (repass)

Es lo que venimos viendo. Produce bytes crudos, del tamaño estrictamente necesario para los tipos elementales. Brilla en colecciones de registros homogéneos: cada registro ocupa exactamente `TAM_REGISTRO` bytes, el archivo se lee con `seek(k · TAM_REGISTRO)` en tiempo constante, y una base de datos de un millón de contactos pesa unos 100 MiB, manipulables sin cargar todo a memoria. Los costos son el diseño explícito del formato, el relleno con ceros en campos de tamaño variable, y la necesidad de controlar manualmente el *endianness*.

7.3. JSON Lines: registros variables en texto portable

El formato **JSON Lines** (también llamado **NDJSON** —*Newline-Delimited JSON*) es un archivo de texto donde cada línea es un documento JSON válido e independiente, separado del siguiente por un salto de línea:

```
# contenido de contactos.jsonl (un JSON por línea)
{"id": 1, "nombre": "Ada Lovelace", "telefono": "4567-1122", "email": "ada@fi
{"id": 2, "nombre": "Alan Turing", "telefono": "4511-8890", "email": "alan@f
{"id": 3, "nombre": "Grace Hopper", "telefono": "4533-2214", "email": "grace@
```

La propiedad clave es que **el archivo se lee y se escribe un registro a la vez**, sin cargar todo el contenido en memoria. Cada línea se parsea independientemente con `json.loads`, y cada registro se escribe con `json.dumps` seguido de un salto de línea:

```
import json

# Agregar un registro al final del archivo (apertura en modo 'a')
def agregar_contacto(ruta, contacto):
    """Agrega un contacto al archivo JSON Lines, al final.

    Precondición: contacto es un diccionario serializable como JSON.
    """
    with open(ruta, 'a', encoding='utf-8') as f:
        linea = json.dumps(contacto, ensure_ascii=False) # sin indentación
        f.write(linea + '\n')                             # cada registro cier

# Recorrer el archivo procesando un registro a la vez
def contar_por_dominio(ruta, dominio):
    """Cuenta cuántos contactos tienen el email en el dominio dado,
    sin cargar el archivo entero en memoria.
```

```

"""
cantidad = 0
with open(ruta, 'r', encoding='utf-8') as f:
    for linea in f:                # iteración línea por línea
        contacto = json.loads(linea)    # parseo independiente
        if contacto['email'].endswith('@' + dominio):
            cantidad += 1
return cantidad

```

El recorrido de JSON Lines es análogo al recorrido de un archivo de texto línea por línea visto en la Semana 6: Python lee una línea completa en cada iteración del `for`, sin necesidad de tener el archivo entero en memoria. Esto vuelve al formato apto para volúmenes de decenas de GiB, imposibles de cargar completos.

JSON Lines es el estándar de facto para logs estructurados (el formato nativo de Elastic, CloudWatch, BigQuery export, las líneas de `jq --jsonl`), para datasets de machine learning (HuggingFace Datasets, OpenAI fine-tuning) y para exportaciones de sistemas de observabilidad. Combina la legibilidad humana de JSON con el procesamiento streaming de los archivos de texto.

7.4. JSON plano: objeto completo, texto portable

El **JSON plano** es lo que se obtiene al guardar *un solo objeto* (diccionario o lista) con `json.dump` en un archivo, típicamente con extensión `.json`. Es el formato apropiado cuando lo que se quiere persistir es un *estado* del programa entre ejecuciones sucesivas: una configuración, preferencias del usuario, el contenido completo de una estructura auxiliar que se recargará entera al próximo arranque.

```

import json

configuracion = {
    'tema': 'oscuro',
    'idioma': 'es-AR',
    'ultimas_búsquedas': ['aleatorio', 'determinista', 'backtracking'],
    'ventana': {'ancho': 1024, 'alto': 768},
}

# Guardar el objeto completo
with open('config.json', 'w', encoding='utf-8') as f:
    json.dump(configuracion, f, ensure_ascii=False, indent=2)

# Leer el objeto completo en otra ejecución del programa
with open('config.json', 'r', encoding='utf-8') as f:
    configuracion = json.load(f)

```

```
print(configuracion['tema']) # 'oscuro'
```

JSON plano no es para bases de datos

Cargar con `json.load` implica leer el archivo entero, parsearlo entero y construir el objeto resultante entero en memoria. Para un archivo de pocos kilobytes o megabytes esto es imperceptible; para un archivo de 500 MiB o de 10 GiB es inaceptable, y en muchas máquinas directamente imposible. Si lo que queremos es persistir muchos registros homogéneos, la herramienta correcta es JSON Lines (texto) o `struct` con registros fijos (binario), nunca JSON plano. Un indicio simple: si el archivo va a crecer sin límite definido con el tiempo, no es caso de JSON plano.

Las virtudes de JSON que justifican su omnipresencia actual son tres: **legible por humanos** (se abre con cualquier editor y se inspecciona a mano), **portable entre lenguajes** (todos los lenguajes modernos lo leen y escriben) y **seguro** (deserializar solo construye tipos básicos: dict, list, str, int, float, bool, None, sin ejecutar código).

7.5. pickle: objeto Python completo, con riesgos

El módulo **pickle** también responde a la Pregunta A —persistir un objeto completo— pero ofreciendo algo que JSON no puede dar: serializa *cualquier* objeto Python, incluidas instancias de clases definidas por el usuario, funciones, árboles, grafos, tensores. El precio es que el formato es binario y específico de Python, por lo que no se puede leer desde otros lenguajes, y que el mecanismo de deserialización permite ejecución de código arbitrario (ver advertencia).

```
import pickle

# Caso típico: checkpoint de un programa que necesita continuar después
estado = {
    'iteracion': 5000,
    'mejor_solucion': [0.37, 0.18, 0.91],
    'historial': [(1, 0.42), (2, 0.41), (3, 0.39)],
}

with open('checkpoint.pkl', 'wb') as f:
    pickle.dump(estado, f)

with open('checkpoint.pkl', 'rb') as f:
    estado = pickle.load(f)    # ¡ATENCIÓN: esto puede ejecutar código!
```

Como JSON plano, pickle **no es para colecciones grandes** que crecen con el tiempo: carga el objeto entero a memoria. Su lugar es el de los objetos complejos pero acotados —un checkpoint de entrenamiento, un cache de resultados intermedios, el estado de una simulación en un momento dado— que el propio programa escribió y va a volver a leer.

Advertencia de seguridad: NO deserializar pickle de fuentes no confiables

La documentación oficial de Python lo dice con letras grandes: *nunca* deserializar un pickle cuya procedencia no sea de absoluta confianza. El formato pickle permite, mediante operandos especiales, reconstruir objetos invocando funciones arbitrarias —incluido `os.system`, `subprocess.Popen` y cualquier otra cosa— lo que significa que abrir un archivo `.pkl` malicioso puede ejecutar código en nuestra máquina. En entornos donde el archivo viene de un tercero o de la red, pickle es un riesgo de seguridad severo. Para esos casos se usa JSON o un formato binario validable como Protocol Buffers, MessagePack o CBOR. Pickle es apropiado solo para persistencia interna de una aplicación, donde el programa mismo es el único origen del archivo.

7.6. Comparación reorganizada: granularidad como eje principal

Técnica	Responde a	Formato	Acceso	Escala	Portabilidad	Seguridad
---------	------------	---------	--------	--------	--------------	-----------

Técnica	Responde a	Formato	Acceso	Escala	Portabilidad	Seguridad
struct + registros fijos	Pregunta B (colección)	Binario, ad-hoc	O(1) al k-ésimo; recorrido secuencial	Excelente (streaming)	Solo con formato acordado	Segura
JSON Lines (.jsonl)	Pregunta B (colección)	Texto, UTF-8	Solo secuencial (sin índice)	Excelente (streaming)	Universal	Segura
JSON plano (.json)	Pregunta A (objeto)	Texto, UTF-8	Todo a memoria	Limitada por la RAM	Universal	Segura
pickle (.pkl)	Pregunta A (objeto)	Binario Python	Todo a memoria	Limitada por la RAM	Solo Python	Riesgosa (ejecuta código)

Guía rápida de elección

Una agenda de contactos con registros que se crean, consultan, modifican y borran: struct con registros fijos (eficiencia, acceso directo) si la uniformidad lo justifica; *slotted pages* con registros variables (Sección 6.2) si las longitudes son muy dispares.

Un log de eventos que crece indefinidamente: JSON Lines si importa la legibilidad y el intercambio con otros sistemas; log binario con prefijo de longitud (Sección 6.1) si importa la compacidad.

Un archivo de configuración o un diccionario que se carga entero al arrancar: JSON plano.

Un checkpoint interno del propio programa que incluye instancias de clases o estructuras complejas: pickle, verificando la procedencia del archivo.

Nunca: JSON plano ni pickle para bases de datos que crecen indefinidamente —son la herramienta equivocada y van a fallar en producción.

8. [Holístico] Conexión con el sistema de archivos del sistema operativo

Cuando nuestro programa Python invoca `open('agenda.dat', 'r+b')` y luego `archivo.seek(97 * k)`,

suceden debajo muchas cosas que el lenguaje oculta detrás de una interfaz aparentemente simple. Comprender ese *debajo* es importante por dos razones. Primero, porque explica por qué ciertas operaciones son rápidas y otras lentas, y por qué a veces los datos no llegan al disco aunque el programa haya terminado sin errores. Segundo, porque evidencia que los patrones que estudiamos esta semana —acceso directo por *offset*, *free list* encadenada, registros de longitud fija— no son invenciones aisladas del programador: son los mismos patrones que el sistema operativo usa para gestionar el propio disco.

8.1. Bloques del sistema de archivos

El sistema operativo no lee ni escribe bytes individuales del disco. Lo hace en unidades llamadas **bloques** (o *clusters* en NTFS), cuyo tamaño típico actual es de **4 KiB** (4096 bytes) en sistemas de archivo modernos como ext4 (Linux), NTFS (Windows) y APFS (macOS). Un archivo de 10000 bytes no ocupa 10000 bytes en disco: ocupa tres bloques completos, es decir, 12288 bytes reservados, de los cuales los últimos 2288 bytes del tercer bloque quedan sin usar.

Cuando pedimos leer el byte 97 de un archivo, el sistema operativo no trae ese byte aislado: trae todo el bloque que lo contiene (los bytes 0 al 4095), lo aloja en memoria principal en una estructura llamada *buffer cache*, y desde allí entrega al programa los bytes solicitados. Esto es transparente para Python, pero tiene dos consecuencias observables:

- **Leer 10 bytes o leer 4000 bytes del mismo bloque cuesta casi lo mismo**, porque el trabajo dominante es transferir el bloque completo desde el medio de almacenamiento.
- **Leer 10 bytes que cruzan la frontera entre dos bloques cuesta el doble**, porque requiere transferir los dos bloques.

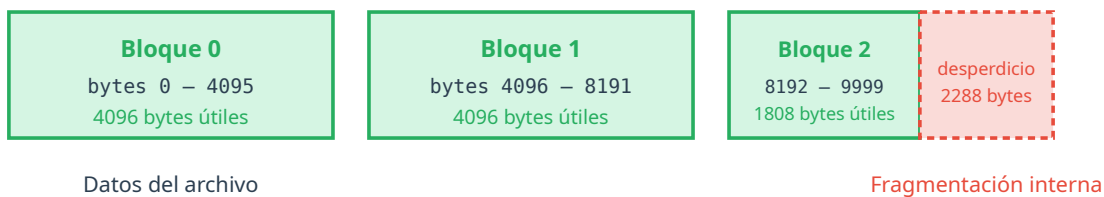
Implicancia de diseño: alinear registros con bloques cuando importa

En nuestra agenda, el registro extendido tiene 97 bytes. En un bloque de 4096 bytes entran $4096 // 97 = 42$ registros completos, y sobran $4096 - 42 \cdot 97 = 22$ bytes del bloque que pertenecen al siguiente registro (que se continúa en el bloque siguiente). Para una agenda de contactos esto no importa; para sistemas con millones de registros y acceso intensivo, los diseñadores eligen tamaños de registro que dividen exactamente el bloque (por ejemplo, registros de 128 bytes: 32 registros por bloque sin restos) o que son múltiplos del bloque (registros de 4096 u 8192 bytes), para que cada lectura de registro use exactamente un número entero de transferencias de bloque.

8.2. Fragmentación interna en disco y el caso SSD

El fenómeno que acabamos de describir —un archivo de 10000 bytes que ocupa 12288 bytes en disco— se llama **fragmentación interna**. Es análogo al fenómeno que ocurre en el *heap* de la Semana 7 cuando el mecanismo de asignación entrega regiones de tamaño redondeado para pedidos de tamaño variable: queda espacio no aprovechado dentro de cada región asignada. En el heap ocurre dentro de la RAM; en el sistema de archivos, dentro del disco.

Archivo de 10 000 bytes en bloques de 4096 bytes



Tamaño lógico: **10 000 bytes** · Tamaño en disco: **12 288 bytes** · Desperdicio: **2 288 bytes (~19 %)**

El desperdicio promedio por archivo es de medio bloque (~2 KiB).

Además de la fragmentación interna existe la **fragmentación externa**: cuando un archivo grande no encuentra un hueco contiguo suficiente y termina repartido en bloques dispersos a lo largo del disco. Durante décadas, este fenómeno fue una preocupación central en el mundo de las computadoras personales, y dio origen a una utilidad tan popular como necesaria: el *desfragmentador*.

La salvedad de 2026: la mayoría de los discos actuales son SSD

La narrativa clásica sobre fragmentación (archivos dispersos que fuerzan al cabezal a dar saltos largos y penalizan el tiempo de acceso) se construyó sobre el supuesto de que el medio de almacenamiento es un **disco duro mecánico** (HDD), con platos giratorios y cabezales móviles. En un HDD, leer dos bloques alejados físicamente cuesta mucho más que leer dos bloques contiguos, porque el cabezal debe desplazarse y esperar a que el sector indicado pase bajo él (latencia rotacional + *seek time*).

Hoy, en cambio, la tecnología predominante en computadoras de escritorio, portátiles y servidores es el **SSD** (*Solid-State Drive*), que almacena los datos en memoria flash NAND y **no tiene partes móviles**. En un SSD, acceder al bloque 10 o al bloque 10 000 000 cuesta prácticamente lo mismo: el tiempo de acceso a cualquier dirección es esencialmente constante. Por lo tanto, *la fragmentación externa no degrada la performance de lectura en un SSD*, y ejecutar un desfragmentador sobre un SSD no solo es inútil sino contraproducente: genera escrituras innecesarias que consumen ciclos de borrado de las celdas flash, cuya cantidad es finita.

La única parte del problema que sobrevive en SSD es el impacto de la fragmentación en los **metadatos** del sistema de archivos. Algunos sistemas —notoriamente NTFS de Windows— registran cada fragmento de un archivo en su tabla maestra (*Master File Table*). Si un archivo se fragmenta mucho (miles de fragmentos no contiguos), los registros de metadatos se inflan y su procesamiento empieza a costar tiempo, aun cuando la lectura física de los datos sea instantánea. Por eso Windows sigue ejecutando una optimización periódica de metadatos en SSDs, distinta del desfragmentado clásico: no reordena bytes para que sean contiguos, sino que compacta las entradas de la tabla de archivos. Los sistemas ext4 y APFS gestionan los metadatos de manera menos sensible a este fenómeno.

La fragmentación *interna* de bloques (el desperdicio dentro del último bloque), en cambio, sigue vigente tanto en HDD como en SSD: es consecuencia de dividir el espacio en unidades

de tamaño fijo y asignar una unidad entera aunque el dato último no la llene.

Conexión con Semana 7: del *heap* y los marcos de página libres a los bloques del disco

La Semana 7 mostró que el *heap* es la región del proceso donde se asignan dinámicamente los objetos a demanda, y que el *garbage collector* de CPython libera automáticamente el espacio que ocupaban los objetos sin referencias. Esa liberación no garantiza que el espacio quede contiguo: tras muchas asignaciones y liberaciones intercaladas, lo que queda libre en el heap es un patrón de huecos de distintos tamaños repartidos entre los objetos aún vivos. El sistema de archivos enfrenta exactamente la misma situación con sus bloques de disco: la liberación de un archivo deja un hueco de bloques libres en medio del espacio ocupado por otros archivos.

El paralelo se vuelve aún más explícito si recordamos el diagrama de paginación de la Semana 7, donde algunos marcos de página aparecían etiquetados como «libres». El kernel del sistema operativo mantiene una **lista de marcos libres** —los marcos de 4 KiB de RAM física que no están asignados a ningún proceso—, y entrega uno cuando un proceso necesita una página nueva. Esa lista de marcos libres es la misma idea estructural que la *free list* que acabamos de implementar para nuestra agenda: un conjunto de unidades de tamaño fijo disponibles para reasignar, devueltas al conjunto cuando se liberan.

8.3. Buffer cache del sistema operativo

Cuando nuestro programa ejecuta `archivo.write(datos)`, los datos *no llegan inmediatamente al disco*. Van primero a una región de memoria RAM llamada **buffer cache** (o *page cache* en Linux), gestionada por el kernel del sistema operativo. El kernel vuelca (*flushes*) los bloques sucios al disco con cierto retraso, agrupando escrituras para optimizar el uso del medio físico. Esta estrategia se llama *write-back*, y tiene una implicancia importante: si el sistema se apaga abruptamente (corte de luz, *kernel panic*) entre el `write` y el vuelco real, los datos escritos se pierden aunque el programa haya terminado sin errores.

Existen dos mecanismos para forzar la persistencia efectiva:

- `archivo.flush()`: vuelca los buffers de Python al kernel (pero los datos pueden quedar todavía en la buffer cache del kernel).
- `os.fsync(archivo.fileno())`: pide al kernel que escriba físicamente los bloques de ese archivo al disco antes de retornar. Este es el llamado fuerte, usado por bases de datos cuando confirman una transacción.

Una analogía entre capas

Python tiene *su* buffer (el que se vuelca con `flush()`); el kernel tiene el *suyo* (la buffer cache,

que se vuelca con `fsync()`; y el propio disco SSD tiene su memoria interna DRAM con datos todavía no grabados en las celdas flash (que se vuelca con un comando ATA *FLUSH CACHE* que el kernel emite periódicamente). Cada capa introduce una demora en pos de eficiencia, y cada capa tiene un mecanismo explícito para forzar la sincronización. Entender esta cadena es indispensable para razonar sobre consistencia de datos, especialmente en sistemas transaccionales.

8.4. `lseek()`: la llamada al sistema detrás de `archivo.seek()`

Cuando escribimos `archivo.seek(97 * k)`, Python termina invocando una *system call* estándar de POSIX llamada `lseek (long seek)`. Esa syscall le dice al kernel: «en el descriptor de archivo correspondiente, actualizá la posición del cursor a este *offset*». Es una operación extremadamente barata: no transfiere datos, solo cambia un entero en la tabla de archivos abiertos del proceso. El costo aparece en la siguiente lectura o escritura, cuando se transfiere el bloque correspondiente.

Es importante distinguir dos cosas que suelen confundirse:

- `seek()` **no lee** del disco. Solo mueve el cursor lógico del archivo.
- La lectura real se dispara en el `read()` o `write()` siguiente, y tiene el costo de traer (o escribir) el bloque completo que contiene ese *offset*.

Esto justifica, retrospectivamente, por qué el acceso directo por *offset* es eficiente para registros de longitud fija: cada `seek() + read(97)` cuesta, en el peor caso, una sola transferencia de bloque desde el medio físico (o cero, si el bloque ya está en la buffer cache por una lectura previa).

8.5. Endianness y *network byte order*

En la Sección 3.3 vimos que un entero de 4 bytes se almacena en memoria con un orden de bytes que depende de la arquitectura. Las arquitecturas modernas de escritorio (x86-64, ARM en modo *little-endian*) son *little-endian*: guardan primero el byte menos significativo. La mayoría de los procesadores RISC tradicionales (y las arquitecturas POWER y SPARC) son, o pueden configurarse como, *big-endian*.

Cuando dos máquinas con *endianness* distinto intercambian datos —por ejemplo, un archivo binario que se copia, o un mensaje que viaja por la red— necesitan un acuerdo. Los protocolos de Internet (TCP, IP, UDP) definen que los campos numéricos de las cabeceras viajan en ***network byte order***, que es ***big-endian***. La historia es que los sistemas Unix tempranos (donde se diseñaron estos protocolos a fines de los años setenta) corrían en hardware *big-endian*; la decisión quedó cristalizada y los procesadores *little-endian* actuales deben convertir los enteros al enviar y al recibir.

Estas funciones de conversión existen en C (`htonl`, `ntohl`, `htons`, `ntohs`) y son la razón por la cual nuestras cadenas de formato de `struct` usan el prefijo `!` cuando escriben datos para red:

```
import struct
```

```
# Cabecera TCP (simplificada): 4 bytes de puerto origen + puerto destino, ambos  
cabecera = struct.pack('!HH', 8080, 443) # ! = network byte order = big-endian  
print(cabecera.hex()) # '1f9001bb' (8080=0x1f90,
```

```
# Si hubiéramos usado '<HH' (little-endian), el archivo enviado se malinterpretaría
```

Conexión con Semana 2: representación binaria de enteros

En la Semana 2 estudiamos cómo se representan los enteros en binario, vimos complemento a dos para negativos, y discutimos por qué un tipo `int32` cubre el rango $[-2^{31}, 2^{31}-1]$. La pregunta que quedó pendiente entonces era: *¿en qué orden se guardan los bytes de ese entero en memoria?* La respuesta llega ahora, porque recién al persistir datos en un archivo —que podrá ser leído por otra máquina— o al enviarlos por la red, la cuestión del *endianness* se vuelve crítica.

8.6. Una analogía unificadora: *free lists* en todas las capas

La estructura de *free list* encadenada que implementamos en la Sección 5 para reutilizar huecos en la agenda no es original nuestra: es una aparición local de un patrón que se repite en casi todos los niveles del sistema:

Capa	Entidad administrada	Dónde se guarda el puntero al siguiente libre
Nuestra agenda (Sección 5)	Registros de 97 bytes	Campo <code>siguiente_libre</code> dentro del registro borrado
Heap del proceso (Semana 7)	Bloques liberados por el GC	Estructura interna del <i>allocator</i> de CPython
Sistema de archivos	Bloques de disco de 4 KiB	Mapa de bits o lista enlazada de bloques libres
Memoria virtual del kernel	Marcos de página (4 KiB)	Listas de marcos libres por zona

En cada capa el problema es el mismo: mantener un conjunto de unidades disponibles para asignar ante demanda, y devolver al conjunto las unidades liberadas. La elección específica del mecanismo (*free list*, mapa de bits, árbol de huecos por tamaño) depende del patrón de uso

esperado, pero la estructura conceptual es siempre la misma. Cuando diseñamos nuestro ABM con *free list* no estamos solo resolviendo el problema de una agenda de contactos: estamos practicando, en pequeña escala y en un dominio visible, el mismo oficio que practican los autores de un *kernel* o de un motor de bases de datos.

Cuatro capas, un mismo patrón

Lo que distingue a un profesional formado de un programador de oficio es, en buena medida, esta capacidad de reconocer la misma estructura actuando en contextos distintos. Quien haya implementado a mano la *free list* de la agenda y haya entendido por qué funciona, tiene en sus manos la idea esencial que explica cómo el sistema operativo gestiona la memoria, cómo el sistema de archivos gestiona los bloques de disco, y cómo una base de datos gestiona sus páginas. El mismo patrón, a distintas escalas.

9. Síntesis y cierre

Esta semana incorporamos una segunda familia de archivos a nuestra caja de herramientas. Los archivos de texto, con los que veníamos trabajando desde la Semana 6, son cómodos para datos que un humano puede querer inspeccionar y para procesos que leen de principio a fin. Los archivos binarios con registros de longitud fija, en cambio, están hechos para volumen, performance y acceso directo: sirven cuando hay muchos datos homogéneos y se necesita llegar al *k*-ésimo registro sin recorrer los anteriores.

El módulo `struct` fue la bisagra: con él pasamos de la manipulación textual de caracteres a la manipulación directa de bytes, con control explícito del tamaño, del orden de bytes y de la codificación de cadenas. A partir de ahí construimos registros de longitud fija, accedimos al *k*-ésimo con `seek(k · TAM_REGISTRO)`, implementamos altas, bajas y modificaciones, y —paso clave— evitamos el desperdicio creciente mediante una *free list* encadenada que reutiliza los huecos dejados por bajas previas. Como alternativa conceptual presentamos el *bitmap* auxiliar de registros libres, la técnica que efectivamente usan los sistemas de archivos reales.

Para los casos donde los registros son intrínsecamente variables, estudiamos dos escenarios canónicos: el *log append-only* con prefijo de longitud, apropiado para datos transaccionales que solo se agregan; y las *slotted pages* con directorio de ranuras y mapa de espacio libre, apropiadas para datos maestros con ABM. Ambos quedaron como menciones conceptuales con propuestas de implementación extracurricular.

El puente con el sistema operativo fue particularmente rico. Vimos que el *seek* que usamos es una *syscall* barata, que la escritura real viaja a través de capas de *buffer cache*, que los archivos ocupan bloques completos (con fragmentación interna inevitable y fragmentación externa hoy casi irrelevante gracias al SSD), y que la *free list* que programamos nosotros es la misma estructura que gestiona el *kernel* para sus bloques libres. El *endianness* y el *network byte order*, que parecían detalles curiosos, se volvieron requisitos operativos cuando los datos cruzan fronteras de máquina.

En el cierre reorganizamos el panorama de formatos de persistencia alrededor de una pregunta central: ¿persisto una colección de registros que crece con el tiempo, o persisto un objeto completo que se carga entero en cada ejecución? A la primera respondemos con `struct`+registros fijos (binario, acceso directo) o con JSON Lines (texto, secuencial, portable); a la segunda, con JSON plano o pickle. Confundir los dos mundos —por ejemplo, usar JSON plano para una base de datos— es la fuente más frecuente de sistemas que funcionan en desarrollo y fallan en producción.

Conexiones del tema



Semanas 1 a 7 (cimientos). Tipos básicos y representación binaria (Semana 2); estructuras de control y funciones; recorridos y búsqueda secuencial (Semanas 3–4); búsqueda binaria y ordenamientos (Semanas 5–6); archivos de texto (Semana 6); punteros, referencias y memoria dinámica en el *heap* (Semana 7).



Semana 8 (este tema). Archivos binarios con `struct`, registros de longitud fija, acceso directo por *offset*, ABM con *free list* (y *bitmap* como alternativa), mención conceptual de registros variables (log append-only y *slotted pages*), serialización (`struct`, JSON Lines, JSON plano, pickle), conexión con el sistema de archivos del SO.



Semana 9 (próxima). Recursividad y manejo de la pila de ejecución (*call stack*); estrategia divide y vencerás revisitada con la lente recursiva; introducción a diccionarios de Python y sus tablas de *hash*.



Semanas 10 a 16. Estructuras de datos (listas enlazadas, pilas, colas, árboles); técnicas algorítmicas (*backtracking*, divide y vencerás, programación dinámica); y proyecto integrador final, en el que los archivos binarios volverán como medio natural de persistencia.



Visión multicapa: qué pasa cuando escribimos un contacto

Cerramos con un ejercicio de síntesis: ordenar las capas que intervienen en una operación tan aparentemente trivial como dar de alta un contacto en nuestra agenda binaria.

Usuario. Escribe un nombre, un teléfono y un email en la interfaz del programa.

Python (nuestro código). Empaqueta los datos con `struct.pack('<Bi i32s16s40s', ...)`, que traduce los enteros a little-endian y codifica las cadenas en UTF-8, produciendo una secuencia de 97 bytes.

Python (intérprete). Invoca los métodos `seek()` y `write()` del objeto archivo, que a su vez llaman a las funciones C `fseek` y `fwrite` del *runtime* estándar.

Biblioteca C. Traduce esas llamadas a las *syscalls* POSIX `lseek` y `write`, invocando al kernel del sistema operativo.

Kernel del SO. Ubica los datos en la *buffer cache*, actualiza los metadatos del archivo (tamaño, fecha de modificación), y agenda el volcado físico para más adelante.

Controlador del disco (SSD). Cuando el volcado ocurre, el *firmware* del SSD traduce la dirección lógica de bloque a direcciones físicas de celdas NAND mediante su tabla de traducción (*FTL*), distribuye las escrituras para prolongar la vida del dispositivo (*wear leveling*) y finalmente programa las celdas.

Seis capas entre el nombre tipeado por el usuario y los electrones atrapados en una celda flash. Cada capa encapsula complejidad y expone una interfaz simple a la de arriba. Esa es, en esencia, la arquitectura de los sistemas informáticos.

Archivos binarios, struct y registros de longitud fija

Semana 8 — Desarrollo de Contenidos — Algoritmia y Programación — Carrera de Informática — FIUBA